

SYSTEM DESIGN HACKATHON

SALESTORM

High-Scale E-Commerce Sale

INDUSTRY USE CASE • DESIGN-FIRST CHALLENGE • AI-ASSISTED CODING

DESIGN FIRST. USE AI TO ACCELERATE. DEFEND EVERY DECISION.

COMPANY USE CASE: SALESTORM E-COMMERCE PLATFORM

Your team is the System Architecture and Engineering consulting unit for a high-volume e-commerce platform. The business is preparing for a limited-stock flash sale in which a sudden surge of customers may place purchase requests at the same time. Management needs a system design that can scale under peak traffic, protect limited inventory, process payments safely, and maintain a correct order lifecycle even when services fail.

The Business Question

How can we handle thousands of simultaneous purchase requests for limited inventory without overselling, while keeping payment and order processing reliable?

The Required Pipeline

Customer → Product Discovery → Cart → Inventory Check → Inventory Reservation → Checkout → Payment → Order → Fulfilment → Shipment → Notification → Delivery Tracking

Hackathon Mission

- Design a scalable high-traffic e-commerce architecture.
- Handle 10,000 concurrent purchase requests for only 100 available units.
- Prevent overselling and duplicate reservations/orders.
- Design reliable payment and order workflows.
- Use AI-assisted coding only as an accelerator for optional prototypes, tests, documentation or simulations.
- Defend architecture, trade-offs and failure-handling decisions.

Team Format

Team Size	Working Style	Final Expectation
3 students	Collaborative, design-first. Every member understands the complete architecture.	A defensible system blueprint, supported by optional AI-assisted prototype/simulation evidence.

Challenge Line: “When thousands of customers compete for limited stock, can your architecture keep every transaction correct?”

1. HACKATHON EVENT SCHEDULE

Date: Monday, 5 October 2026

Participants: III Year Students

Venue: Respective Class Rooms

Time: 9:00 AM – 4:00 PM

Time	Activity
9:00 AM – 9:30 AM	Registration and Team Formation
9:30 AM – 9:45 AM	Hackathon Guidelines and SALESTORM Problem Statement Briefing
9:45 AM – 11:15 AM	Hackathon – Phase I: Industry Problem Analysis and High-Level System Design
11:15 AM – 11:35 AM	Refreshment Break
11:35 AM – 12:25 PM	Hackathon – Phase II: High Concurrency and Inventory Reservation Design
12:25 PM – 1:15 PM	Hackathon – Phase III: Payment Reliability and Order Management Design
1:15 PM – 2:00 PM	Lunch Break
2:00 PM – 2:45 PM	Hackathon – Phase IV: Low-Level Design, SOLID and Design Patterns
2:45 PM – 3:15 PM	Hackathon – Phase V: Stress Scenarios and Jury Challenge
3:15 PM – 3:30 PM	Final Design Preparation and Submission
3:30 PM – 3:50 PM	Final 5-Minute Pitch and Technical Evaluation
3:50 PM – 4:00 PM	Results, Feedback and Valedictory Session

Design-First Principle

The primary evaluation is the quality of the system design. Production implementation is not required.
A lightweight prototype or AI-assisted simulation may be used only to validate critical design assumptions.

2. BUSINESS CONTEXT & SUCCESS CRITERIA

SALESTORM represents a modern e-commerce flash-sale environment. During high-demand campaigns, customer traffic can increase sharply within a short period. Limited-stock products create simultaneous purchase requests, while payment gateways, databases and downstream services may experience delays or failures. The business therefore requires an architecture that balances scalability, availability, performance and data consistency.

Business Objectives

- Support sudden increases in customer traffic without system collapse.
- Prevent inventory overselling under concurrent purchase requests.
- Provide temporary inventory reservation with expiry and release.
- Ensure safe and idempotent payment processing.
- Maintain a consistent and traceable order lifecycle.
- Recover gracefully from service, database and payment failures.
- Produce a design that an engineering team can understand, implement and operate.

Key Success Criteria

Success Criterion	Expected Outcome
Concurrency	10,000 simultaneous requests can be handled through controlled contention and defined scaling behaviour.
Inventory	100 available units cannot result in more than 100 successful sales.
Reservation	Unpaid or expired reservations are released correctly.
Payment	Duplicate payment requests do not create duplicate transactions.
Order	Successful purchases eventually reach a valid order state.
Reliability	Defined recovery paths exist for service and gateway failures.
Scalability	Architecture supports horizontal scaling during peak demand.

Reference Solution Architecture

Users → CDN/WAF → Load Balancer → API Gateway → Product / Cart / Sale Services → Inventory / Reservation → Checkout → Payment → Order → Shipment / Notification

Supporting infrastructure may include caching, message queues, databases, monitoring, logging and distributed tracing. Teams may propose alternatives if the trade-offs are justified.

3. INDUSTRY SCENARIO, DATA & DATABASE DESIGN

The system must support customers, products, inventory, promotions, carts, reservations, payments, orders and fulfilment. The database design should reflect the business lifecycle and support the concurrency requirements of the flash sale.

Critical Industry Scenario: Product X has 100 units available. 10,000 customers simultaneously select “Buy Now”. The design must determine which requests successfully reserve inventory, ensure inventory never becomes negative, and provide correct outcomes for successful, failed, expired and duplicate requests.

Core Database Entities

Entity	Purpose
CUSTOMER	Customer identity and account information
PRODUCT	Product catalogue information
CATEGORY	Product classification
INVENTORY	Available, reserved and sold quantities
INVENTORY_RESERVATION	Temporary reservation, status, expiry and idempotency
CART / CART_ITEM	Customer shopping cart
ORDER / ORDER_ITEM	Confirmed purchase and line items
PAYMENT	Payment transaction and state
SALE / DEAL / COUPON	Sale and promotion rules
SHIPMENT	Fulfilment and delivery information
NOTIFICATION	Customer communication events

Concurrency-Critical Data

Field	Purpose
inventory_id	Unique inventory record
product_id	Product reference
available_quantity	Units available for reservation
reserved_quantity	Units currently reserved
sold_quantity	Units confirmed as sold
version	Concurrency/version control
updated_at	Last update timestamp
reservation_id	Unique reservation reference
expires_at	Reservation expiry time

Field	Purpose
idempotency_key	Duplicate request prevention

Database Design Requirement: Students must address primary keys, foreign keys, indexes, constraints, transaction boundaries, concurrency control, consistency and audit information.

4. SYSTEM DESIGN TASKS – WHAT YOU MUST DO

This is a **design-first** hackathon. The primary task is to reason about architecture, interfaces, data flow, state, trade-offs, scalability and failure recovery. Coding is optional and should support—not replace—the design.

Stage 1 – Requirement Analysis

- Translate the business problem into functional requirements.
- Define measurable non-functional requirements: throughput, latency, availability, consistency, security and recovery.
- State assumptions, constraints, expected traffic and critical dependencies.
- Identify which requirements are strict guarantees and which are targets.

Stage 2 – Architecture Design

- Create a System Context Diagram.
- Create a Container / Service Architecture.
- Create a Component Diagram for critical services.
- Create a Deployment Diagram.
- Identify synchronous versus asynchronous communication.
- Show API Gateway, Load Balancer, Cache, Database, Message Queue and External Systems.
- Identify bottlenecks and explain why each major component exists.

Stage 3 – Concurrency & Inventory Design

- Design a concurrency-safe inventory reservation mechanism.
- Define reservation creation, confirmation, expiry and release.
- Prevent overselling when multiple customers request the last available units.
- Handle duplicate reservation requests.
- Compare at least two concurrency-control approaches and justify the selected approach.
- Define transaction boundaries and consistency guarantees.

Stage 4 – Payment & Order Design

- Design payment success, failure and timeout flows.
- Prevent duplicate payments through idempotency.
- Define retry, timeout and circuit-breaker behaviour where appropriate.
- Design payment success followed by Order Service failure.
- Define valid order states, transitions, cancellation and recovery.

5. SYSTEM DESIGN TASKS – CONTINUED

Stage 5 – Low-Level Design

Select at least three critical modules, preferably **Inventory, Payment and Order**. The objective is good object-oriented design, not a large codebase.

- Create class diagrams showing responsibilities, relationships and interfaces.
- Create sequence diagrams for purchase/reservation, payment and order workflows.
- Create a state diagram for order or reservation lifecycle.
- Identify service boundaries and domain responsibilities.
- Show where validation, idempotency and error handling belong.

Stage 6 – SOLID & Design Patterns

- Map SOLID principles to actual classes/components.
- Use appropriate patterns such as Strategy, Factory, State, Observer, Adapter, Facade, Repository and Circuit Breaker.
- Explain the problem solved by each selected pattern and the trade-off introduced.
- Show how the design can accommodate a new payment provider, pricing strategy or delivery partner with minimal change.

Stage 7 – API & Event Design

- Define API method, endpoint, request, response, status, authentication and error handling.
- Identify commands, events and asynchronous messages.
- Use idempotency where repeated requests could cause duplicate business transactions.
- Define event ownership and consumer responsibilities.

Stage 8 – Architecture Decisions & Trade-offs

- Record major decisions in an Architecture Decision Record (ADR).
- Compare alternatives such as SQL vs. NoSQL, synchronous vs. asynchronous processing, optimistic vs. pessimistic concurrency and cache strategies.
- State why the selected option is appropriate for SALESTORM.

AI-Assisted Coding – Permitted Use

AI-assisted coding is allowed for optional prototype development, test-data generation, simulation scripts, API stubs, boilerplate code, documentation and test cases. The team remains responsible for correctness and must be able to explain every generated component.

6. DESIGN-FIRST AI-ASSISTED CODING POLICY

What AI May Be Used For

- Generate boilerplate code for a small prototype.
- Create API skeletons, DTOs, model classes or repository stubs.
- Generate unit-test cases or concurrency test data.
- Create load-test scripts or simple simulation scripts.
- Generate documentation drafts or UML text such as PlantUML/Mermaid.
- Explore alternative implementation approaches after the architecture has been designed.

What AI Must NOT Replace

- Requirement analysis.
- Architecture selection and system boundaries.
- Concurrency reasoning.
- Inventory consistency decisions.
- Payment and order state modelling.
- Trade-off analysis.
- Security and reliability decisions.
- Final technical explanation by the student.

AI Usage Expectations

Requirement	Expected Practice
Transparency	Mention the AI tool(s) used and purpose.
Validation	Review, test and correct AI-generated code before demonstrating it.
Ownership	Every team member must understand the generated artefact.
Design First	Architecture and design decisions must be produced before relying on generated implementation.
Evidence	If AI-assisted artefacts are submitted, include a short AI-use note or prompt summary.

Important: The jury evaluates engineering judgement, not the amount of code generated by AI.

Recommended AI-Assisted Workflow

Problem Understanding → Requirements → HLD → Critical LLD → Trade-offs → AI-Assisted Prototype/Simulation → Validation → Architecture Revision → Final Defense

7. CONCURRENCY, INVENTORY & PAYMENT CHALLENGES

Challenge A – High Concurrency

Given: **10,000 customers** and **100 available units**.

- Explain how traffic enters and is distributed.
- Identify where requests are throttled, queued or rejected.
- Identify the exact point at which inventory contention is controlled.
- Explain horizontal scaling and state management.
- Explain duplicate request and idempotency handling.

Challenge B – Inventory Reservation

A correct design should support:

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

Failure paths:

RESERVED → PAYMENT_FAILED → RELEASED
RESERVED → TIMEOUT → RELEASED

Students must explain how successful reservations never exceed available inventory.

Challenge C – Payment Reliability

Scenario	Required Design Response
Payment succeeds	Confirm payment and proceed to order creation.
Payment fails	Release reservation and update customer/order state.
Payment times out	Use safe retry/reconciliation without duplicate payment.
Duplicate payment request	Use idempotency and a unique transaction reference.
Payment succeeds; Order Service fails	Use event/retry/reconciliation and compensation where necessary.

Challenge D – Order Lifecycle

CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

8. PRACTICAL DESIGN CHALLENGE

The practical challenge is primarily a **design walkthrough**. Teams may use an AI-assisted prototype or simulation to validate selected assumptions, but implementation depth is not the scoring focus.

Practical Test Case

Stock = 100 | Concurrent users = 10,000 | Payment success = 95% | Payment failures = 5% | Duplicate requests = 2% | Order Service unavailable for 30 seconds

Student Demonstration

- Trace one successful purchase from request arrival to order confirmation.
- Trace one failed payment and show reservation release.
- Trace a duplicate Buy request and show idempotent behaviour.
- Trace payment success followed by temporary Order Service failure.
- Show what happens when inventory reaches zero.
- Explain what changes when traffic increases by 50×.
- Explain recovery when the database or payment gateway fails.

Optional AI-Assisted Validation

Teams may create a small simulation or test harness using AI-assisted coding to demonstrate concurrency, idempotency or reservation expiry. The simulation is evidence for the design; it is not a substitute for the architecture.

Jury Design Questions

- Why is this the correct service boundary?
- Where exactly is inventory consistency guaranteed?
- What happens if two requests reach the inventory service at the same time?
- Why did you select this concurrency strategy?
- Why is this operation synchronous while another is asynchronous?
- How does the design recover from payment success followed by order failure?
- Which component is the likely bottleneck and how will it scale?

9. LLD, SOLID, DESIGN PATTERNS & RELIABILITY

Low-Level Design Expectations

- Class diagram for at least three critical modules.
- Interface-based design where appropriate.
- Sequence diagram for purchase and reservation.
- Sequence diagram for payment.
- Sequence diagram for order creation and recovery.
- State diagram for order or reservation.

SOLID Principle Mapping

Principle	Expected Demonstration
SRP	Separate payment processing, order management and notification.
OCP	Add new pricing or payment strategies without changing core logic.
LSP	Subtypes remain substitutable for abstractions.
ISP	Use focused interfaces rather than large interfaces.
DIP	High-level services depend on abstractions rather than concrete providers.

Suggested Design Patterns

Pattern	Possible Application
Strategy	Pricing, payment selection or delivery strategy
Factory	Creating payment/provider objects
State	Order and reservation lifecycle
Observer	Order events and notifications
Adapter	External payment or delivery providers
Repository	Persistence abstraction
Facade	Checkout orchestration
Circuit Breaker	Protection from failing external services

Reliability & Failure Handling

- Service timeout and retry policy.
- Circuit breaker for unstable external dependencies.
- Dead-letter handling for failed asynchronous messages.
- Idempotent processing of repeated messages.
- Compensation or reconciliation for distributed workflow failures.
- Monitoring, logging and tracing for critical transactions.

10. SECURITY, SCALABILITY & OBSERVABILITY

Security Requirements

- Authentication and authorization.
- HTTPS and secure API communication.
- Rate limiting and abuse protection.
- Input validation.
- Secure handling of payment information.
- Audit logging and secrets management.

Scalability Requirements

- Normal traffic: approximately 10,000 requests/sec.
- Flash-sale traffic: architecture should reason about up to 500,000 requests/sec.
- Horizontal scaling, caching, queueing, load balancing and database scaling should be considered.
- Students must identify likely bottlenecks and explain how the architecture addresses them.

Observability Requirements

- Define key metrics: request rate, latency, error rate, inventory reservation failures, payment failures and order conversion.
- Define structured logs for critical business events.
- Define distributed tracing across checkout, payment and order workflows.
- Identify alerts for inventory inconsistency, payment failures and queue backlog.

Architecture Trade-off Requirement

A strong solution should explicitly state what it sacrifices or complicates—for example consistency vs. availability, synchronous simplicity vs. asynchronous resilience, or database simplicity vs. horizontal scalability.

11. EXPECTED OUTPUT – WHAT A GOOD SOLUTION LOOKS LIKE

Your final outcome is a **design blueprint**. Implementation is optional. A good solution clearly demonstrates how the architecture responds to the business problem and why the selected design is appropriate.

Expected System Design Outcome

- Complete HLD showing major services and infrastructure.
- Detailed LLD for critical modules.
- Concurrency-safe inventory reservation mechanism.
- Reliable payment and order workflows.
- Clear SOLID and design-pattern application.
- Database and API design aligned with business workflows.
- Scalability and fault-tolerance strategy.
- Security and observability considerations.
- Clear technical justification and trade-offs.

Example Architecture Recommendation

Users → CDN/WAF → Load Balancer → API Gateway → Product / Cart / Sale Service → Inventory & Reservation → Checkout → Payment → Order → Shipment / Notification

Supporting infrastructure may include cache, message broker, database cluster, monitoring, logs and distributed tracing. This is an example only; teams may propose alternatives with sound justification.

Example Recommendation – Last Item Scenario

If one unit remains and two customers attempt to purchase it concurrently, the design should provide a deterministic concurrency mechanism so that only one request successfully reserves the unit. The other request receives an appropriate out-of-stock or reservation-failed response.

Design Quality Indicators

- Clear separation of responsibilities.
- Minimal unnecessary coupling.
- Explicit state transitions.
- Well-defined consistency boundaries.
- Graceful failure behaviour.
- Scalable bottleneck management.
- Simple and explainable architecture.

12. EXPECTED FILES / DELIVERABLES

No	Deliverable	Mandatory?
1	Requirements & Assumptions Document	Yes
2	System Context Diagram	Yes
3	HLD Architecture	Yes
4	Container Diagram	Yes
5	Component Diagram	Yes
6	Deployment Diagram	Yes
7	Database / ER Diagram	Yes
8	Class Diagram	Yes
9	Purchase / Reservation Sequence Diagram	Yes
10	Payment Sequence Diagram	Yes
11	Order Sequence Diagram	Yes
12	Order / Reservation State Diagram	Yes
13	SOLID Mapping	Yes
14	Design Pattern Mapping	Yes
15	API Specification	Yes
16	Scalability & Reliability Design	Yes
17	Security & Observability Design	Yes
18	Architecture Decision Record	Yes
19	AI-Assisted Prototype / Simulation Evidence	Optional
20	AI Usage Note / Prompt Summary	If AI used
21	Final Presentation	Yes

The deliverables should demonstrate the design before the code. If a prototype is submitted, it should be small, targeted and directly linked to a design assumption or practical scenario.

13. TOOLS, TEAM ROLES, SUBMISSION & EVALUATION

Recommended Tools

Purpose	Suggested Tools
Architecture / UML	Draw.io / diagrams.net, PlantUML, Mermaid, StarUML, Visual Paradigm
Database	MySQL Workbench, dbdiagram.io
API	Swagger / OpenAPI, Postman
Collaboration	GitHub, GitLab, Google Drive
Simulation / Validation	JMeter, Locust, Postman Collection Runner
AI-Assisted Coding	ChatGPT, GitHub Copilot, Gemini, Claude or institution-approved AI tools
Documentation	README, Markdown, PPT/PDF

Suggested Team Ownership

Student Role	Primary Ownership
Student 1 – System Architect	HLD, architecture, scalability and deployment
Student 2 – LLD & Design Engineer	LLD, SOLID, UML and design patterns
Student 3 – Data & API Engineer	Database, APIs, events and integration
Student 4 – Reliability Engineer	Concurrency, failure handling, security and recovery

All team members may be questioned on any module during evaluation.

14. EVALUATION FRAMEWORK

The evaluation gives significantly greater weight to **system design and engineering judgement** than to implementation.

Area	Weight
Business & Requirement Understanding	10%
High-Level Architecture & Service Boundaries	15%
High Concurrency & Scalability Design	15%
Inventory Reservation & Consistency	15%
Payment & Order Management Design	10%
LLD & UML	10%
SOLID & Design Patterns	5%
Database, API & Event Design	5%
Reliability, Security & Observability	5%
Architecture Trade-offs & Defense	5%
AI-Assisted Prototype / Validation	3%
Final Pitch & Team Understanding	2%
TOTAL	100%

Evaluation Philosophy

- Design correctness is more important than code volume.
- Students receive credit for clear reasoning even without a prototype.
- AI-generated code receives credit only when validated and correctly connected to the design.
- Every team member must be able to defend architecture decisions.
- Alternative architectures are acceptable when trade-offs are clearly justified.

15. SUBMISSION FOLDER STRUCTURE & FINAL PITCH

```
SALESTORM_TEAM_NAME/  
01_Requirements/  
02_HLD/  
03_LLD/  
04_Database/  
05_API/  
06_SOLID/  
07_Design_Patterns/  
08_Scalability_Reliability/  
09_Security_Observability/  
10_ADR/  
11_AI_Assisted_Validation/ (optional)  
12_Presentation/  
README.md
```

Final 5-Minute Pitch

Time	Content
30 sec	Problem – industry need and SALESTORM challenge
30 sec	Requirements – functional, non-functional and constraints
60 sec	HLD – complete architecture and major decisions
60 sec	Critical Design – 10,000 customers competing for 100 units
45 sec	Payment & Order – success, failure and recovery
45 sec	LLD + SOLID + Patterns – one critical module
30 sec	Scalability, Reliability and AI-Assisted Validation

Final Jury Question: Your architecture has 100 units remaining and 10,000 customers are simultaneously clicking Buy Now. Walk us through exactly what happens from request arrival until the final valid orders are confirmed.

Build a system design that an industry engineering team can understand, challenge, implement and operate—not just a collection of diagrams.

16. PRACTICAL EVALUATION – DESIGN PERFORMANCE

The practical evaluation focuses on whether students can apply System Design concepts to realistic industry conditions. **Complete application implementation is not mandatory.** Design walkthroughs, modelling and optional AI-assisted simulations are sufficient.

Practical Activity	Marks
1. High-Concurrency HLD & Architecture Defense	25
2. Inventory Reservation LLD & Consistency Design	25
3. Payment + Order Workflow Design	25
4. Stress Scenarios, Trade-offs & Architecture Defense	25
TOTAL	100

Practical Evaluation – Key Checks

- Trace the request path for 10,000 concurrent purchase attempts.
- Show how 100 units are protected from overselling.
- Demonstrate or logically walk through reservation expiry and release.
- Demonstrate payment failure and recovery.
- Demonstrate payment success followed by Order Service failure.
- Demonstrate duplicate request handling using idempotency.
- Explain what changes when traffic increases by 50×.
- Explain recovery when the database or payment gateway fails.
- Explain how AI-assisted code, if used, was validated against the architecture.

Industry-Oriented Expected Outcome

At the end of the hackathon, each team should produce a defensible, scalable and implementation-ready system blueprint for a high-traffic e-commerce flash-sale platform.

- Translate an industry problem into functional and non-functional requirements.
- Design HLD and LLD using appropriate architectural and object-oriented principles.
- Handle concurrency and inventory consistency.
- Design reliable payment and order workflows.
- Apply SOLID principles and design patterns to real modules.
- Design APIs, databases, events and service interactions.
- Reason about scalability, failure recovery, security and observability.
- Use AI-assisted coding responsibly to validate selected design assumptions.
- Defend architecture decisions using technical evidence and clear trade-offs.

The intended outcome is a coherent system design that can withstand realistic industry scenarios and can be handed to an implementation team as an architectural blueprint. The hackathon rewards design thinking, engineering judgement and technical defense—not the volume of AI-generated code.